

FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA
FACULDADE DE ENGENHARIA, TECNOLOGIA E INOVAÇÃO
COORDENAÇÃO DOS CURSOS DE CIÊNCIA DA COMPUTAÇÃO

Disciplina: Programação Orientada a Objetos

Professor: Dr. Lucas Marques da Cunha

SIAPE: 3269899

Aluno(a): _____

Nota: _____

Aluno(a): _____

Avaliação Prática – Sistema de Cadastro de Clientes 2.0

Nesta avaliação será utilizado o sistema de cadastro de clientes:

<https://github.com/lucasmarquesc/CadastroDeClientes>

O sistema já possui interface gráfica, tabela, cadastro, exclusão, atualização, leitura e gravação em arquivo, além de acesso ao banco de dados SQLite.

O objetivo desta avaliação é modificar o sistema existente, aplicando os conceitos de expressões regulares, tratamento de exceções, arquivos, banco de dados e manipulação de datas.

Bloco 1 – Validação de Dados com Regex (2,0 pontos)

Neste bloco, você deverá implementar validações utilizando expressões regulares antes de salvar ou atualizar um cliente.

Questão 1 – Validação do Nome (0,5 ponto)

Implemente uma validação para o campo nome.

O nome deve aceitar apenas letras, espaços e acentos.

Exemplo de expressão regular:

```
String regexNome = "[A-Za-zÀ-ÿ ]+$";
```

O sistema deve rejeitar nomes contendo números ou caracteres especiais.

Questão 2 – Validação do E-mail (0,5 ponto)

Implemente uma validação para o campo e-mail.

Exemplo de expressão regular:

```
String regexEmail = "[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
```

O sistema deve rejeitar e-mails sem arroba, sem domínio ou com formato inválido.

Questão 3 – Validação do Telefone (0,5 ponto)

Implemente uma validação para telefone no formato:

(69) 99999-9999

Exemplo de expressão regular:

```
String regexTelefone = "^\\(\\d{2}\\)\\s?\\d{4,5}-\\d{4}$";
```

Observação: Crie métodos auxiliares para validar os dados, evitando repetição de código. A validação deverá ser utilizada tanto no cadastro quanto na atualização de clientes.

Questão 4 – Mensagens ao Usuário (0,5 ponto)

Caso algum dado seja inválido, o sistema deverá exibir uma mensagem informando o problema. As mensagens deverão ser exibidas em uma janela modal.

Bloco 2 – Tratamento de Exceções (2,0 pontos)

Neste bloco, você deverá melhorar o tratamento de exceções do sistema.

Atualmente, várias exceções são apenas exibidas no console por meio de `printStackTrace()`. O objetivo é tratar os erros de forma mais adequada e apresentar mensagens compreensíveis ao usuário.

Questão 1 – Exceções de Banco de Dados (0,5 ponto)

Modifique as operações do DAO para tratar possíveis falhas de conexão, inserção, atualização, exclusão e listagem de clientes. O sistema deve exibir mensagens amigáveis quando ocorrer erro no banco de dados. As mensagens devem ser exibidas utilizando uma janela modal.

Questão 2 – Tratamento na Importação/Exportação de Arquivos (0,5 ponto)

Ao importar um arquivo CSV, o sistema deverá tratar:

- arquivo inexistente;
- arquivo vazio;
- linha com quantidade incorreta de campos;
- erro de leitura.

Ao exportar um relatório, o sistema deverá tratar possíveis erros de gravação. Caso o arquivo não possa ser salvo, uma mensagem deverá ser exibida ao usuário.

Questão 4 – Validação de Seleção na Tabela (0,5 ponto)

Corrija as ações que dependem de uma linha selecionada na tabela.

Por exemplo, ao tentar excluir ou atualizar sem selecionar um cliente, o sistema deverá exibir:

Selecione um cliente antes de continuar.

Bloco 3 – Alteração da Tabela com Campo Data (2,5 pontos)

Neste bloco, você deverá adicionar ao sistema o campo **data de cadastro**.

Esse campo deverá ser preenchido automaticamente com a data atual no momento em que o cliente for cadastrado.

Questão 1 – Alteração da Classe Cliente (0,5 ponto)

Adicione o atributo `dataCadastro` na classe `Cliente`.

Utilize a classe `LocalDate`.

```
import java.time.LocalDate;

private LocalDate dataCadastro;
```

Exemplo de Uso do LocalDate

Utilize `LocalDate.now()` para obter a data atual.

Exemplo:

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

LocalDate hoje = LocalDate.now();

DateTimeFormatter formato =
    DateTimeFormatter.ofPattern("dd/MM/yyyy");

String dataFormatada = hoje.format(formato);

System.out.println(dataFormatada);
```

Para converter uma `String` do banco de dados para `LocalDate`, utilize:

```
LocalDate data = LocalDate.parse("2026-06-23");
```

Para salvar no banco de dados, utilize:

```
stmt.setString(5, cliente.getDataCadastro().toString());
```

Questão 2 – Alteração do Banco de Dados (0,5 ponto)

Altere a tabela `clientes` para incluir a coluna `data_cadastro`.

Exemplo de comando SQL:

```
ALTER TABLE clientes
ADD COLUMN data_cadastro TEXT;
```

Caso prefira recriar a tabela, utilize:

```
CREATE TABLE clientes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
    telefone TEXT NOT NULL,
    email TEXT NOT NULL,
    sexo TEXT NOT NULL,
    data_cadastro TEXT NOT NULL
);
```

Questão 3 – Alteração do ClienteDAO (0,5 ponto)

Modifique os métodos `salvar`, `listar` e `atualizar` para considerar o novo campo `data_cadastro`.

Ao cadastrar um novo cliente, a data deve ser gerada automaticamente.

```
Cliente cliente = new Cliente(
    nome,
    telefone,
    email,
    sexo,
    LocalDate.now()
);
```

Questão 4 – Alteração da JTable (0,5 ponto)

Modifique o `ClienteTableModel` para exibir a nova coluna:

Data de Cadastro

A tabela deverá apresentar cinco colunas:

- a) Nome;
- b) Telefone;
- c) Email;
- d) Sexo;
- e) Data de Cadastro.

Bloco 4 – Menu Ferramentas (2,0 pontos)

Neste bloco, você deverá substituir o menu **Preferências** pelo menu **Ferramentas**.

O novo menu deverá conter as seguintes opções:

```
JMenu mnFerramentas = new JMenu("Ferramentas");
menuBar.add(mnFerramentas);

JMenuItem mntmValidar = new JMenuItem("Validar_dados");
JMenuItem mntmExportar = new JMenuItem("Exportar_relatorio");
JMenuItem mntmImportar = new JMenuItem("Importar_CSV_validado");

mnFerramentas.add(mntmValidar);
mnFerramentas.add(mntmExportar);
mnFerramentas.add(mntmImportar);
```

Questão 1 – Menu Exportar Relatório (0,5 ponto)

Ao clicar em:

Ferramentas > Exportar relatório

o sistema deverá gerar um arquivo de relatório em formato `.txt`.

O relatório deverá conter:

- título do relatório;
- total de clientes;
- quantidade de clientes masculinos;
- quantidade de clientes femininos;
- listagem dos clientes cadastrados;
- data de geração do relatório.

Resultado esperado:

RELATRIO DE CLIENTES

Data de gerao: 23/06/2026

Total de clientes: 4

Masculino: 2

Feminino: 2

Nome: Joo Silva

Telefone: (69) 99999-9999

Email: joao@email.com

Sexo: Masculino

Data de cadastro: 23/06/2026

Caso o arquivo seja gerado corretamente, o sistema deverá exibir:

Relatório exportado com sucesso.

Caso ocorra erro:

Erro ao exportar relatório.

Questão 2 – Menu Importar CSV Validado (0,5 ponto)

Ao clicar em:

Ferramentas > Importar CSV validado

o sistema deverá abrir um seletor de arquivos e permitir a escolha de um arquivo CSV.

Cada linha do arquivo deverá ser validada antes de ser adicionada à tabela.

Formato esperado do CSV:

Nome,Telefone,Email,Sexo

Joo Silva,(69) 99999-9999,joao@email.com,Masculino

Maria Souza,(69) 98888-7777,maria@email.com,Feminino

Linhas inválidas deverão ser ignoradas.

Resultado esperado:

Importao concluida.

Registros importados: 10

Registros rejeitados: 3

Caso o arquivo esteja vazio:

0 arquivo selecionado está vazio.

Caso o arquivo possua formato inválido:

0 arquivo contém linhas em formato inválido.

Questão 3 – Integração com o Banco de Dados (0,5 ponto)

Os clientes importados pelo CSV validado deverão ser salvos também no banco de dados.

Após a importação, a tabela da interface deverá ser atualizada com os dados presentes no banco.

Questão 4 – Organização do Código (0,5 ponto)

O código deverá ser organizado em métodos auxiliares.

Exemplos:

```
private boolean validarCliente(Cliente cliente) {  
    // valida nome, telefone e email  
}  
  
private void exportarRelatorio(File arquivo) {  
    // gera o arquivo de relatorio  
}  
  
private void importarCSVValidado(File arquivo) {  
    // importa e valida as linhas do CSV  
}
```

Bloco 5 – Consultas ao Banco de Dados (2,0 pontos)

Neste bloco, você deverá modificar o sistema para que as consultas sejam realizadas diretamente no banco de dados.

O objetivo é evitar buscas apenas na lista carregada em memória e utilizar comandos SQL para recuperar informações do banco SQLite.

Questão 1 – Busca por Nome no Banco de Dados (1,0 ponto)

Atualmente, a funcionalidade de busca utiliza os dados presentes na tabela carregada em memória.

Modifique o sistema para que a pesquisa seja realizada diretamente no banco de dados.

Crie um método no DAO semelhante ao exemplo abaixo:

```
public ArrayList<Cliente> buscarPorNome(String nome)
```

Sugestão de consulta SQL:

```
SELECT *  
FROM clientes  
WHERE nome LIKE ?;
```

Exemplo de utilização:

```
stmt.setString(1, "%" + nome + "%");
```

Resultado esperado:

- Ao pesquisar por “Joao”, deverão ser retornados todos os clientes cujo nome contenha a palavra João.
- A JTable deverá ser atualizada com o resultado da consulta.

Exemplo:

Pesquisa: Joao

Resultados encontrados: 3

Joao Silva
Joao Pedro
Maria Joao

Caso nenhum registro seja encontrado:

Nenhum cliente encontrado.

Questão 2 – Criação dos Componentes de Filtro por Data (0,25 ponto)

Adicione à interface gráfica os componentes necessários para filtrar clientes por data de cadastro.

O filtro poderá utilizar:

- duas caixas de texto;
- dois campos formatados;
- ou qualquer outra solução adequada.

Formato esperado:

dd/MM/yyyy

Exemplo:

01/06/2026 até 30/06/2026

Questão 3 – Consulta por Intervalo de Datas (0,5 ponto)

Crie um método no DAO responsável por consultar clientes cadastrados dentro de um intervalo de datas.

Exemplo:

```
public ArrayList<Cliente>
buscarPorPeriodo(
    LocalDate dataInicial,
    LocalDate dataFinal
)
```

Sugestão de consulta SQL:

```
SELECT *
FROM clientes
WHERE data_cadastro
BETWEEN ? AND ?;
```

Resultado esperado:

Periodo selecionado:

01/06/2026

ate

30/06/2026

Clientes encontrados: 15

Observação: Após realizar a busca por período, a JTable deverá exibir apenas os registros retornados pela consulta.

Não será permitido realizar filtros apenas na lista em memória.

Os dados deverão ser recuperados diretamente do banco de dados.

Questão 4 – Tratamento de Erros (0,25 ponto)

Implemente validações para:

- data inicial vazia;
- data final vazia;
- formato inválido;
- data inicial maior que a data final.

Exemplos de mensagens esperadas:

Informe as duas datas para realizar a consulta.

Data inicial não pode ser maior que a data final.

Formato de data inválido. Utilize dd/MM/yyyy.

Critérios Gerais de Avaliação

Além dos itens específicos de cada bloco, serão considerados:

- funcionamento correto do sistema;
- organização do código;
- clareza das mensagens exibidas ao usuário;
- uso adequado de orientação a objetos;
- ausência de erros de execução;
- tratamento adequado de exceções;
- integração correta entre interface, modelo, DAO e banco de dados.